

MET CS688

WEB ANALYTICS AND MINING

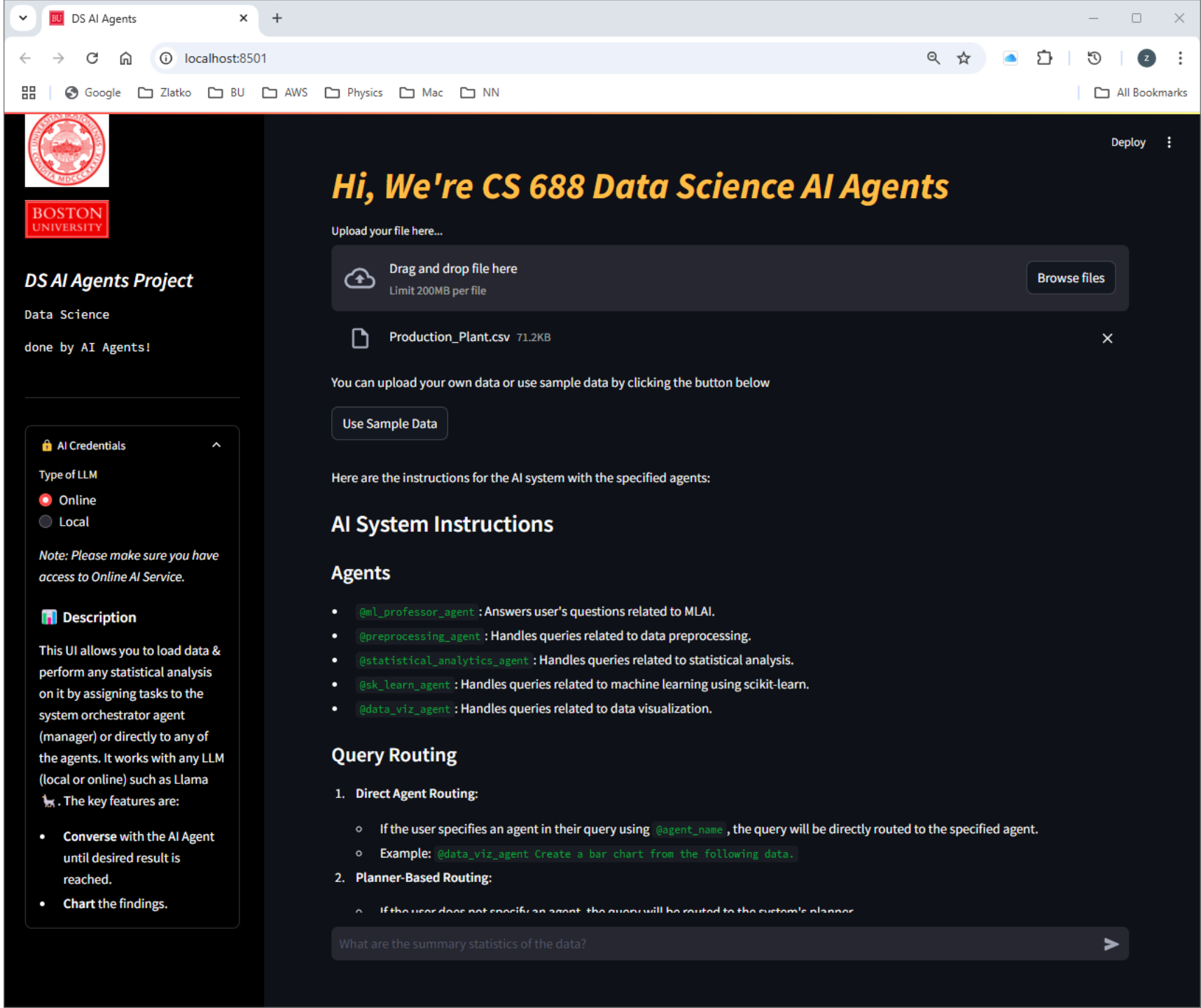
ZLATKO VASILKOSKI

DATA SCIENCE AI AGENTS

DS AI Agents

CS 688 class AI-powered set of data scientist/programmer agents.

- Works with any LLM
  - Local LLM (data security & free)
  - Online AI Service (GPT, Claude etc.)
- Integrate your own functions as new AI Agents skills.
- Built using the professional DSPy framework.
- Assign tasks to the system orchestrator agent (manager) or directly to the agents.
- Load data & perform any statistical analysis on it with
  - Detailed explanations of the statistical methods used
  - Interactive charts, tables, etc.
  - Generated and executed Python code for the analysis.



CS 688 DS AI Agents - Description

The CS 688 DS-Analyst is an AI-driven data analytics agentic system designed to simplify and enhance the data science process.

- By integrating various specialized AI agents, this tool aims to make complex data analysis tasks more accessible and efficient for data analysts and scientists.
- Auto-Analyst provides a streamlined approach to data preprocessing, statistical analysis, machine learning, and visualization, all within an interactive Streamlit interface.

Agents with Data Science Specialty:

- **A BU MLAI Professor Agent:** Answers user's questions related to MLAI.
- **Preprocessing Agent**: Handles data cleaning, transformation, and preparation tasks.
- **Statistical Analytics Agent**: Performs comprehensive statistical analyses and generates descriptive statistics.
- **Data Visualization Agent**: Generates a wide range of Plotly & seaborn charts and visualizations.
- **Python Programmer Agent**: Integrates with Scikit-Learn library to build and evaluate machine learning models.

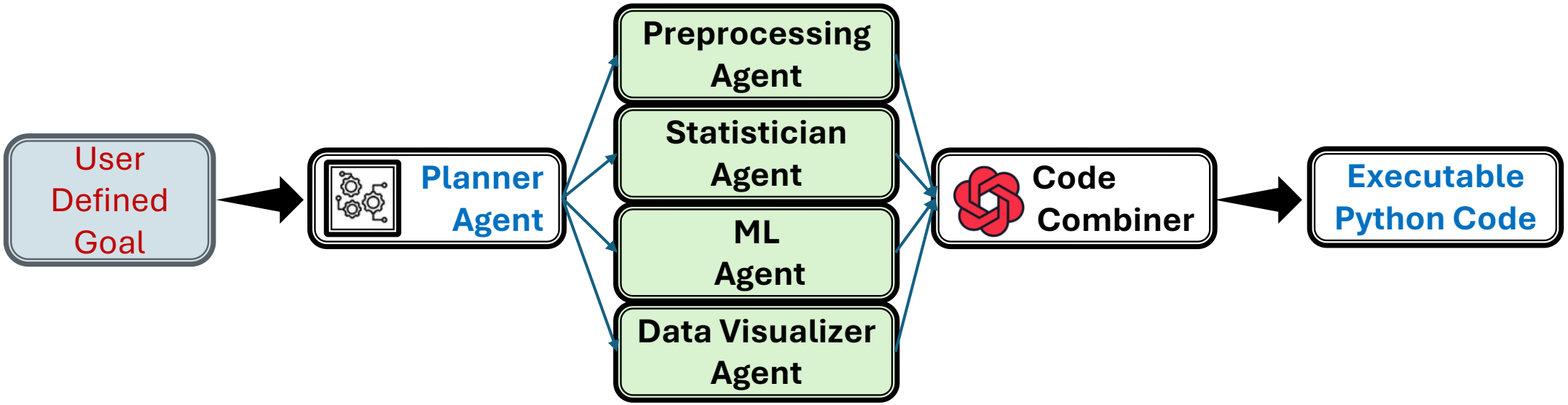
Key Files and Purpose

1. **agents.py**:
   - **Description**: Contains the definitions for various AI agents used in the system.
   - **Key Agents**:
     - auto_analyst_ind: Routes queries to the appropriate agent based on user input and provides a detailed response.
     - auto_analyst: Integrates a planner agent for routing queries and a code combiner agent for synthesizing outputs from multiple agents.
2. **memory_agents.py**:
   - **Description**: Defines agents that help summarize memory and errors.
   - **Key Agents**:
     - memory_summarize_agent: Provides summaries of agent responses and user goals.
     - error_memory_agent: Analyzes and summarizes code errors and suggested corrections.
3. **retrievers.py**:
   - **Description**: Contains functions and configurations for retrieving and processing data.
   - **Key Functions**:
     - return_vals: Collects useful information about data columns, such as statistics and top categories.
     - correct_num: Cleans numeric columns by removing commas and converting to float.
     - make_data: Pre-processes data and generates a description of the dataset.
   - **Styling Instructions**: Provides instructions for styling Plotly charts for different types of visualizations, including line charts, bar charts, histograms, pie charts, and heat maps.
4. **ds_agents_app.py**:
   - **Description**: The main Streamlit script that runs the application and integrates all the agents and functionalities provided in the other files.

Design Workflow

The system begins with a user-defined goal.

- The **planner agent** then delegates tasks to a group of worker agents, each responsible for generating code to address a specific part of the problem.
  - **The planner agent could delegate tasks to some of the agents, not necessarily to all.**
  - The **planner agent** takes three inputs, **user-defined goal**, the **datasets available**, and **agents' descriptions**.
  - It outputs a plan in this format: Agent1-> Agent2-> Agent3….
- Finally, all the individual pieces of code are gathered and integrated by a code combiner agent, resulting in a single, cohesive script that accomplishes the entire goal.



Description of Agents

Analysis Agents

- Most analysis agents share a common structure with slight variations in their prompts.
- They accept two inputs:
  - the user-defined goal and
  - the dataset index.
- They produce two outputs which can be useful for debugging or redirecting the agent:
  - the code for the analysis and
  - commentary

Code Combiner Agent

- The purpose of this agent is to clean the output from all the agents into one coherent script. It takes a long string of a list of code, and outputs code.

Optional Agents/Indexes

For the agent to work more smoothly and to catch some errors, there are additional agent or indexes.

Query1: What are the summary statistics of the Housing dataset?

The App generates

- Numeric summary
- Categorical summary
- Correlation Matrix
- Chart Price vs Area
- Python code for all this

Correlation Matrix:

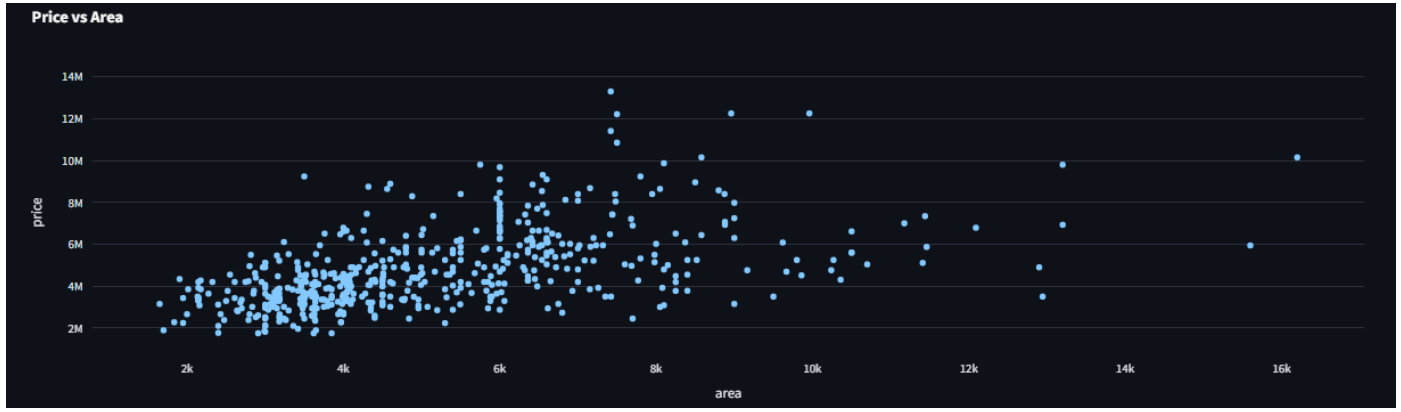
	price	area	bedrooms	bathrooms	stories	parking
price	1	0.536	0.3665	0.5175	0.4207	0.3844
area	0.536	1	0.1519	0.1938	0.084	0.353
bedrooms	0.3665	0.1519	1	0.3739	0.4086	0.1393
bathrooms	0.5175	0.1938	0.3739	1	0.3262	0.1775
stories	0.4207	0.084	0.4086	0.3262	1	0.0455
parking	0.3844	0.353	0.1393	0.1775	0.0455	1

Numerical Summary Statistics:

	price	area	bedrooms	bathrooms	stories	parking
count	545	545	545	545	545	545
mean	4,766,729.2477	5,150.5413	2.9651	1.2862	1.8055	0.6936
std	1,870,439.6157	2,170.141	0.7381	0.5025	0.8675	0.8616
min	1,750,000	1,650	1	1	1	0
25%	3,430,000	3,600	2	1	1	0
50%	4,340,000	4,600	3	1	2	0
75%	5,740,000	6,360	3	2	2	1
max	13,300,000	16,200	6	4	4	3

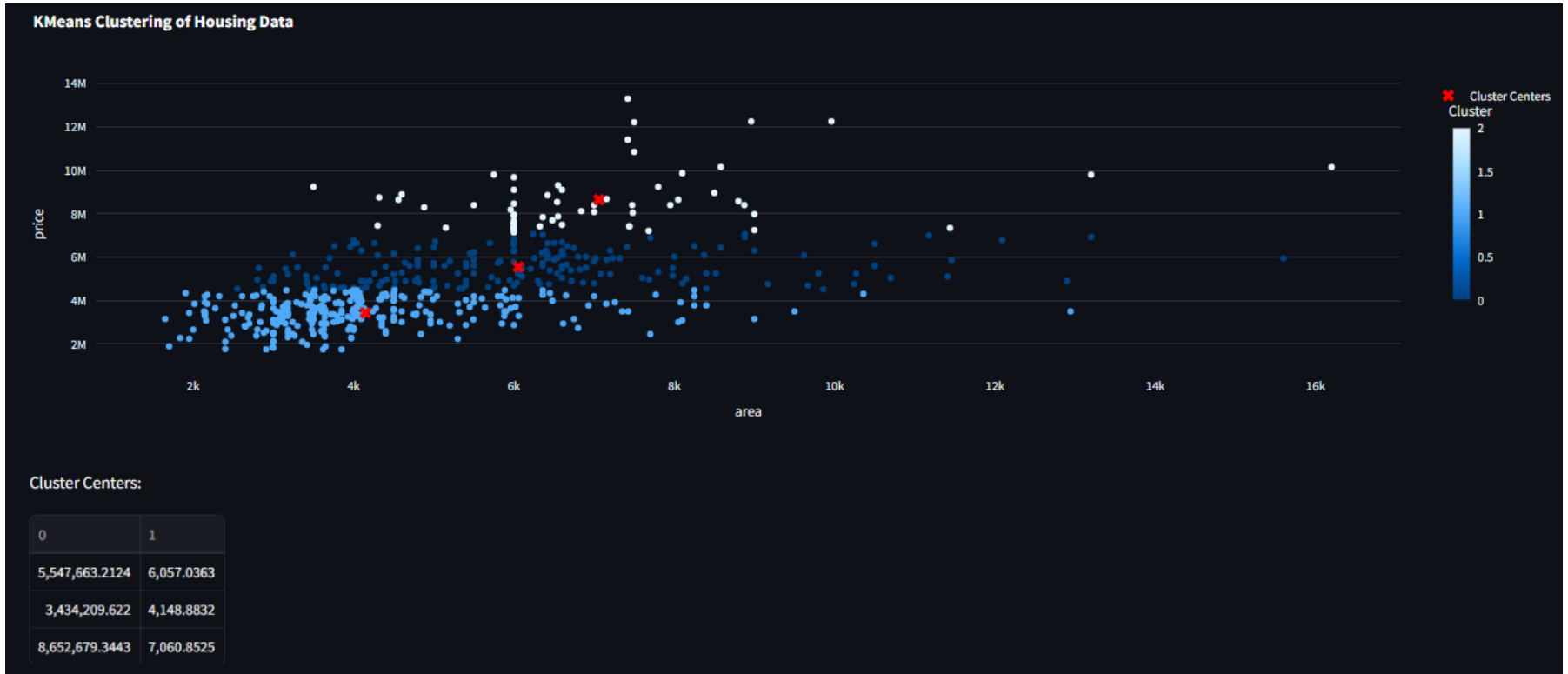
Categorical Summary Statistics:

	mainroad	guestroom	basement	hotwaterheating	airconditioning	prefarea	furnishingstatus
count	545	545	545	545	545	545	545
unique	2	2	2	2	2	2	3
top	yes	no	no	no	no	no	semi-furnished
freq	468	448	354	520	373	417	227



Query2: @sk_learn_agent can you find clusters using KMeans and visualize them using Plotly for price and area? Please describe KMeans clustering.

- **Reply on what is KMeans**: In this analysis, we utilized KMeans clustering to group the housing data based on price and the number of bedrooms. KMeans is an unsupervised learning algorithm that partitions the dataset into K distinct clusters based on feature similarity. The algorithm iteratively assigns data points to the nearest cluster center and updates the centers until convergence. The resulting visualization helps to identify patterns and relationships within the data, providing insights into how price and bedroom count correlate across different clusters.
- **Summary of Performed Analysis**: The code performs KMeans clustering on the housing dataset based on the 'price' and 'area' features. It visualizes the resulting clusters using a scatter plot, where each point represents a house, and the colors indicate different clusters. The cluster centers are also displayed to provide insight into the average characteristics of each cluster. This analysis helps in understanding how houses are grouped based on their price and area, which can be useful for market segmentation and pricing strategies.



Cluster Centers:

0	1
5,547,663.2124	6,057.0363
3,434,209.622	4,148.8832
8,652,679.3443	7,060.8525

Query3: @statistical_analytics_agent please build a OLS model for price, with area, bedrooms, bathrooms and stories as independent variables. Also describe what OLS model is.

The App Replies on what is Ordinary Least Squares (OLS):

- The OLS model is a statistical method used to estimate the relationships between one dependent variable and one or more independent variables.
- It assumes that the relationship between the variables is linear and aims to minimize the sum of the squared differences between the observed and predicted values.
- In this case, we are using OLS to understand how the features of a house (area, number of bedrooms, bathrooms, and stories) influence its price.
- The output will provide coefficients for each variable, indicating their impact on the price, along with statistical significance measures.

OLS Regression Results

Dep. Variable:	price	R-squared:	0.536
Model:	OLS	Adj. R-squared:	0.532
Method:	Least Squares	F-statistic:	155.7
Date:	Sun, 22 Sep 2024	Prob (F-statistic):	1.68e-88
Time:	18:47:08	Log-Likelihood:	-8434.5
No. Observations:	545	AIC:	1.688e+04
Df Residuals:	540	BIC:	1.690e+04
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	-2.821e+05	2.52e+05	-1.118	0.264	-7.78e+05	2.14e+05
area	380.0244	25.864	14.693	0.000	329.219	430.830
bedrooms	1.983e+05	8.51e+04	2.330	0.020	3.11e+04	3.65e+05
bathrooms	1.2e+06	1.22e+05	9.871	0.000	9.61e+05	1.44e+06
stories	5.316e+05	7.08e+04	7.511	0.000	3.93e+05	6.71e+05

Omnibus:	94.932	Durbin-Watson:	1.043
Prob(Omnibus):	0.000	Jarque-Bera (JB):	232.104
Skew:	0.895	Prob(JB):	3.97e-51
Kurtosis:	5.649	Cond. No.	2.64e+04

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.64e+04. This might indicate that there are strong multicollinearity or other numerical problems.

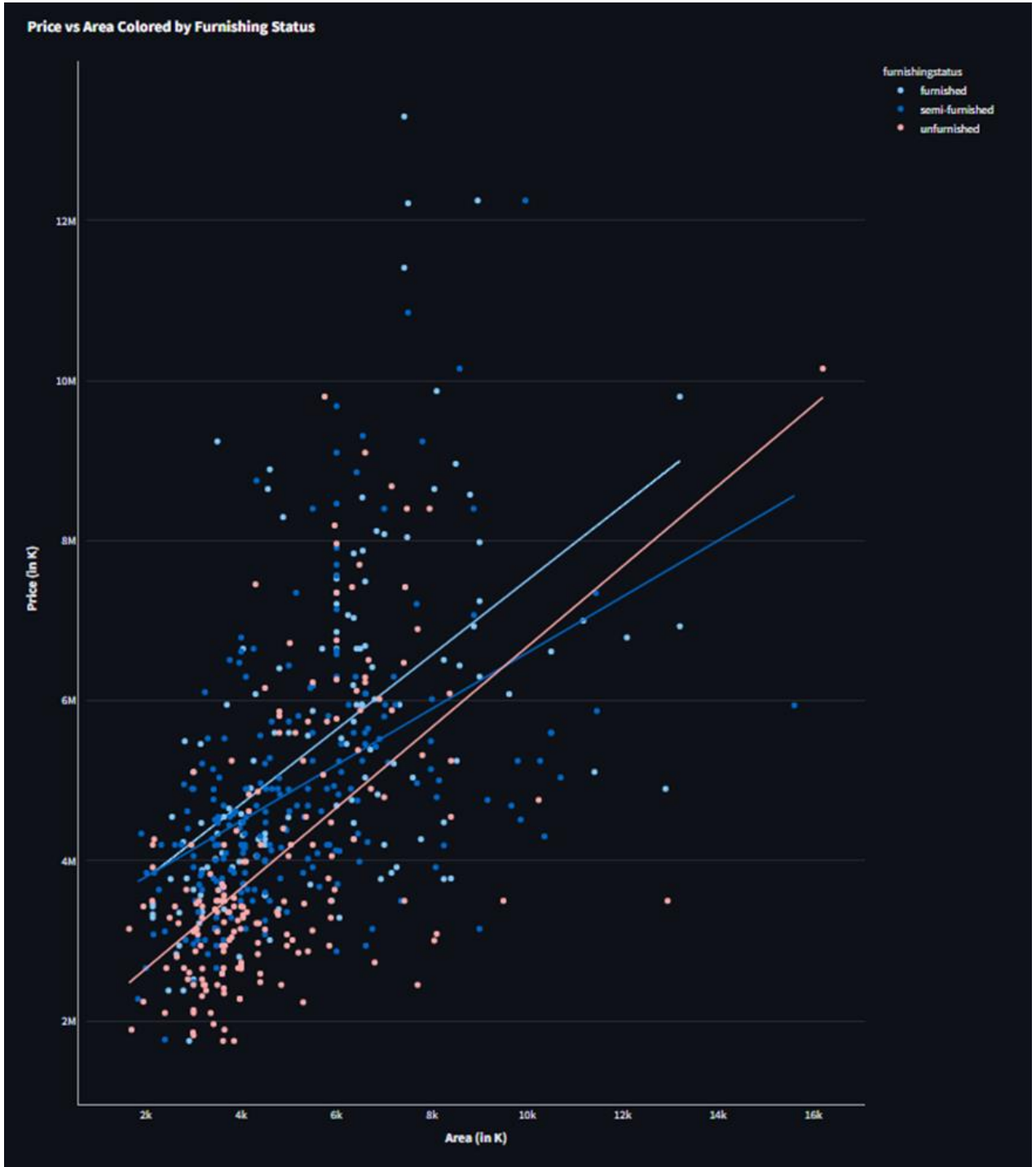
Query4: @data_viz_agent visualize the last analysis as Plotly scatter plot for price and area colored by furnishing Status.

Note:

- The app uses its **memory** to identify what was its previous task and uses that context to accomplish this task.

The App Replies:

- The scatter plot effectively visualizes the relationship between the area of the houses and their prices.
- Different colors representing the furnishing status.
- The trendline helps to identify any potential correlation between the two variables, providing insights into how area influences price in the housing market.



The Prompt & the Code for the Goal Refiner Agent

```
class goal_refiner_agent(dspy.Signature):
    # Called to refine the query incase user query not elaborate
    """You take a user-defined goal given to a AI data analyst planner agent,
    you make the goal more elaborate using the datasets available and agent_desc"""
    dataset = dspy.InputField(desc="Available datasets loaded in the system, use this df,columns  set df as copy of df")
    Agent_desc = dspy.InputField(desc="The agents available in the system")
    goal = dspy.InputField(desc="The user defined goal ")
    refined_goal = dspy.OutputField(desc='Refined goal that helps the planner agent plan better')

```

The Prompt & the Code for the Storyteller Agent

```
class story_teller_agent(dspy.Signature):
    # Optional helper agent, which can be called to build a analytics story
    # For all of the analysis performed
    """ You are a story teller agent, taking output from different data analytics agents, you compose a compelling story for what was done """
    agent_analysis_list = dspy.InputField(desc="A list of analysis descriptions from every agent")
    story = dspy.OutputField(desc="A coherent story combining the whole analysis")
```

The Prompt & the Code for the Analytical Planner Agent

```python
class analytical_planner(dspy.Signature):
    # The planner agent which routes the query to Agent(s)
    # The output is like this Agent1->Agent2 etc
    """ You are data analytics planner agent. You have access to three inputs
    1. Datasets
    2. Data Agent descriptions
    3. User-defined Goal
    You take these three inputs to develop a comprehensive plan to achieve the user-defined goal from the data & Agents available.
    In case you think the user-defined goal is infeasible you can ask the user to redefine or add more description to the goal.

    Give your output in this format:
    plan: Agent1->Agent2->Agent3
    plan_desc = Use Agent 1 for this reason, then agent2 for this reason and lastly agent3 for this reason.

    You don't have to use all the agents in response of the query

    """
    dataset = dspy.InputField(desc="Available datasets loaded in the system, use this df,columns  set df as copy of df")
    Agent_desc = dspy.InputField(desc="The agents available in the system")
    goal = dspy.InputField(desc="The user defined goal ")
    plan = dspy.OutputField(desc="The plan that would achieve the user defined goal", prefix='Plan:')
    plan_desc = dspy.OutputField(desc="The reasoning behind the chosen plan")

```

The Prompt & the Code for the Data Viz Agent

```python
class data_viz_agent(dspy.Signature):
    # Visualizes data using Plotly
    """
    You are AI agent who uses the goal to generate data visualizations in Plotly.
    You have to use the tools available to your disposal
    If row_count of dataset > 50000, use sample while visualizing
    use this
    if len(df)>50000:
        .......
    Only this agent does the visualization
    Also only use x_axis/y_axis once in update layout
    {dataset}
    {styling_index}

    You must give an output as code, in case there is no relevant columns, just state that you don't have the relevant information

    Make sure your output is as intended! DO NOT OUTPUT THE DATASET/STYLING INDEX
    ONLY OUTPUT THE CODE AND COMMENTARY. ONLY USE ONE OF THESE 'K','M' or 1,000/1,000,000. NOT BOTH

    You may be give recent agent interactions as a hint! With the first being the latest
    DONT INCLUDE GOAL/DATASET/STYLING INDEX IN YOUR OUTPUT!
    You can add trendline into a scatter plot to show it changes,only if user mentions for it in the query!
    You are logged in streamlit use st.write instead of print

    """
    goal = dspy.InputField(desc="user defined goal which includes information about data and chart they want to plot")
    dataset = dspy.InputField(
        desc=" Provides information about the data in the data frame. Only use column names and dataframe_name as in this context")
    styling_index = dspy.InputField(desc='Provides instructions on how to style your Plotly plots')
    code = dspy.OutputField(
        desc="Plotly code that visualizes what the user needs according to the query & dataframe_index & styling_context")
    commentary = dspy.OutputField(
        desc="The comments about what analysis is being performed, this should not include code")

```

The Prompt & the Code for the Code Fixer Agent

```python
class code_fix(dspy.Signature):
    # Called to fix unexecutable code
    """
    You are an AI which fixes the data analytics code from another agent, your fixed code should only fix the faulty part of the code, rest should remain the same
    You take the faulty code, and the error generated and generate the fixed code that performs the exact analysis the faulty code intends to do
    You are also give user given context that guides you how to fix the code!


    please reflect on the errors of the AI agent and then generate a
   correct step-by-step solution to the problem.
   You are logged in streamlit use st.write instead of print


    """
    faulty_code = dspy.InputField(desc="The faulty code that did not work")
    previous_code_fixes = dspy.InputField(desc="User adds additional context that might help solve the problem")
    error = dspy.InputField(desc="The error generated")

    faulty_code = dspy.OutputField(desc="Only include the faulty code here")
    fixed_code = dspy.OutputField(desc="The fixed code")
```

Using the Iris Dataset

- A

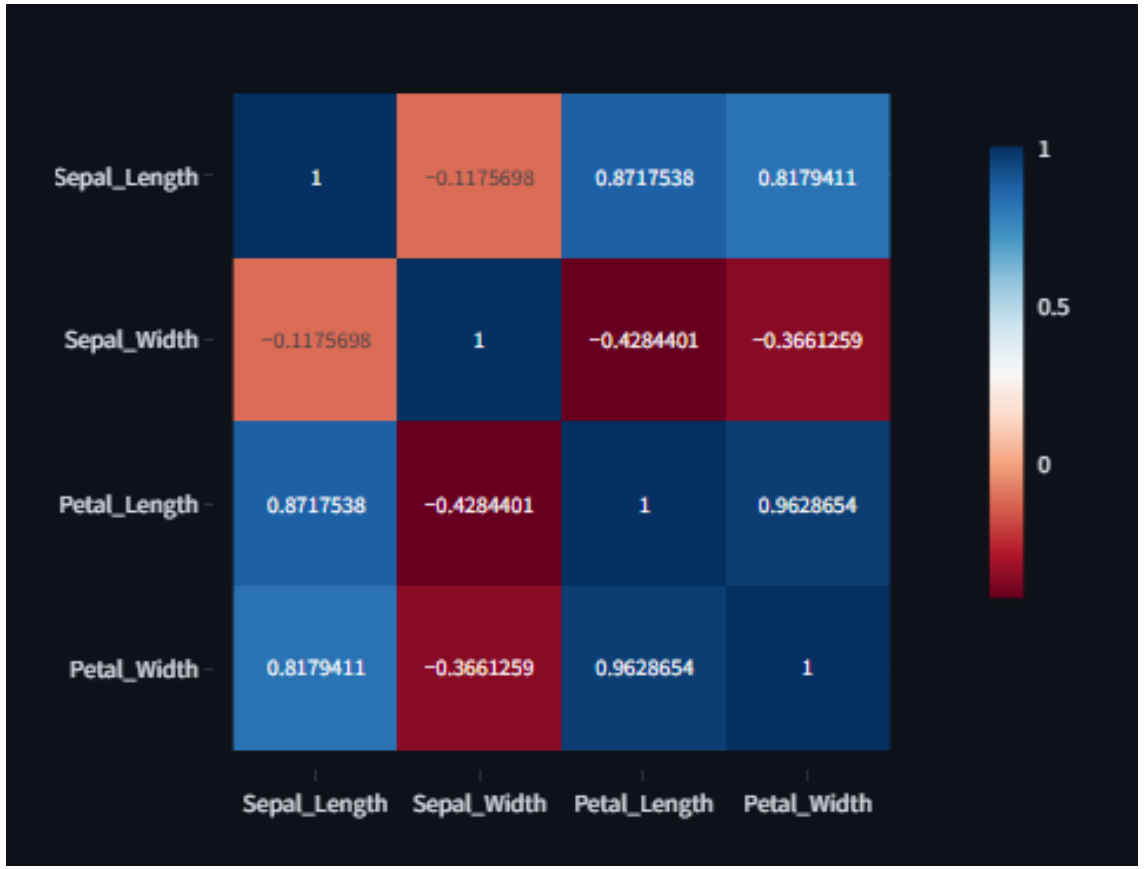
Query1: What are the summary statistics of the Iris dataset?

The App generates

- Numeric & Categorical summary
- Correlation Matrix
- Python code for all this

Summary Statistics:

	Sepal_Length	Sepal_Width	Petal_Length	Petal_Width
count	150	150	150	150
mean	5.8433	3.0573	3.758	1.1993
std	0.8281	0.4359	1.7653	0.7622
min	4.3	2	1	0.1
25%	5.1	2.8	1.6	0.3
50%	5.8	3	4.35	1.3
75%	6.4	3.3	5.1	1.8
max	7.9	4.4	6.9	2.5



Query2: @sk_learn_agent can you find clusters using KMeans and visualize them using Plotly for Sepal_Length and Sepal_Width? Please describe KMeans clustering.

- **Reply on what is KMeans**: sk_learn_agent[commentary]: In this analysis, we utilized KMeans clustering to identify groups within the Iris dataset based on the sepal dimensions. The resulting clusters are visualized in a scatter plot, where each point represents an observation colored by its assigned cluster. The cluster centers are also displayed, providing insight into the average sepal dimensions for each cluster. This method is effective for understanding the natural groupings in the data, which can be useful for classification tasks or exploratory data analysis.



Cluster Centers:

0	1
6.8128	3.0745
5.7736	2.6925
5.006	3.428

Query3: @statistical_analytics_agent please build a OLS model for Sepal_Length, with Sepal_Width, Petal_Length, Petal_Width as independent variables. Also describe what OLS model is.

The statistical_analytics_agent Replies:

- The analysis performed here is an Ordinary Least Squares (OLS) regression, which is a method used to estimate the relationships between a dependent variable and one or more independent variables.
- The OLS model minimizes the sum of the squares of the residuals (the differences between observed and predicted values) to find the best-fitting line through the data.
- This model will help us understand how changes in Sepal_Width, Petal_Length, and Petal_Width affect Sepal_Length in the Iris dataset.

OLS Regression Results

Dep. Variable:	Sepal_Length	R-squared:	0.859
Model:	OLS	Adj. R-squared:	0.856
Method:	Least Squares	F-statistic:	295.5
Date:	Wed, 13 Nov 2024	Prob (F-statistic):	8.59e-62
Time:	12:03:25	Log-Likelihood:	-37.321
No. Observations:	150	AIC:	82.64
Df Residuals:	146	BIC:	94.69
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	1.8560	0.251	7.401	0.000	1.360	2.352
Sepal_Width	0.6508	0.067	9.765	0.000	0.519	0.783
Petal_Length	0.7091	0.057	12.502	0.000	0.597	0.821
Petal_Width	-0.5565	0.128	-4.363	0.000	-0.809	-0.304

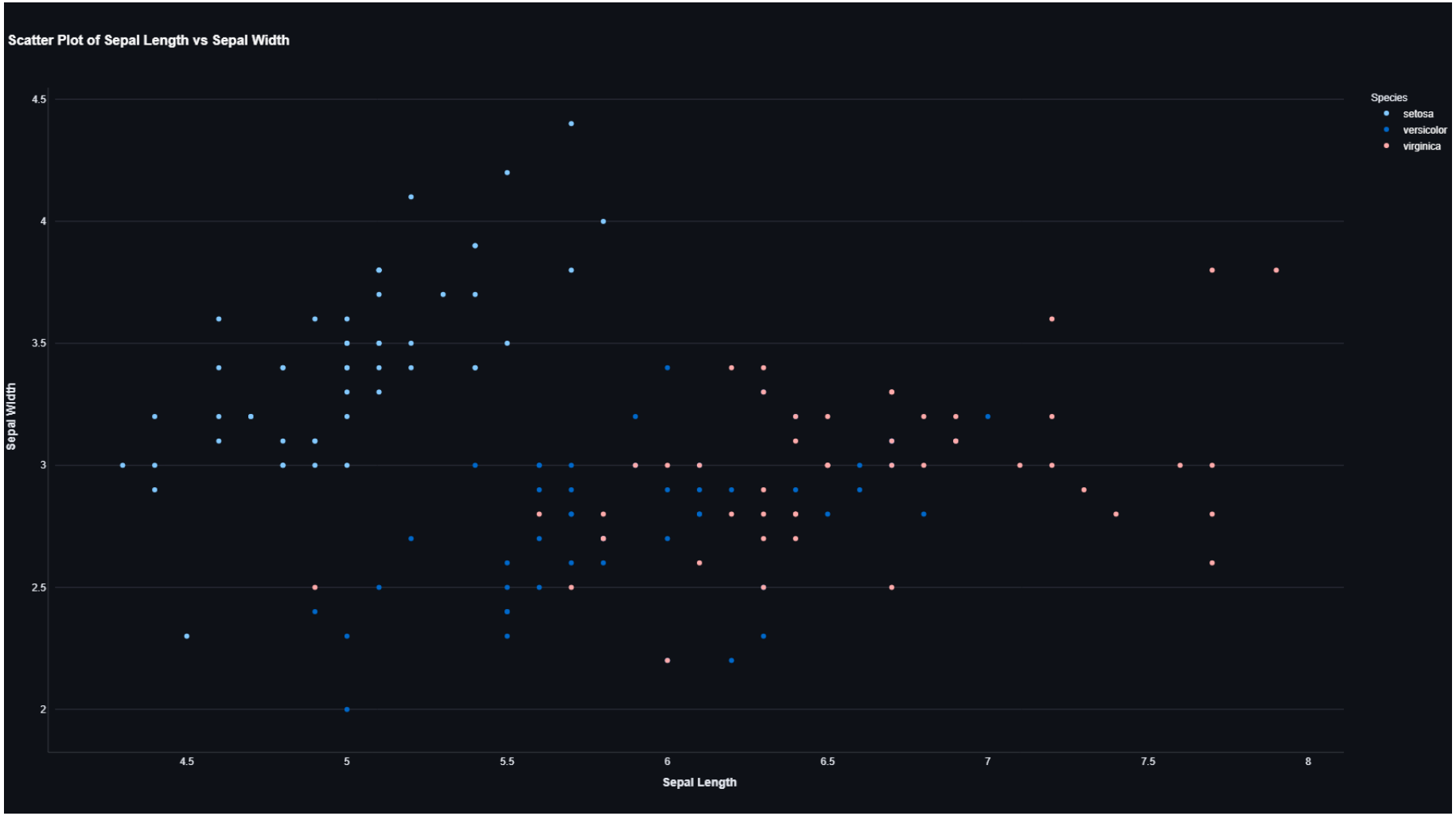
Omnibus:	0.345	Durbin-Watson:	2.060
Prob(Omnibus):	0.842	Jarque-Bera (JB):	0.504
Skew:	0.007	Prob(JB):	0.777
Kurtosis:	2.716	Cond. No.	54.7

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Query4: @data_viz_agent visualize the loaded dataset as Plotly scatter plot for Sepal_Length and Sepal_Width colored by Species.

The data_viz_agent Replies:

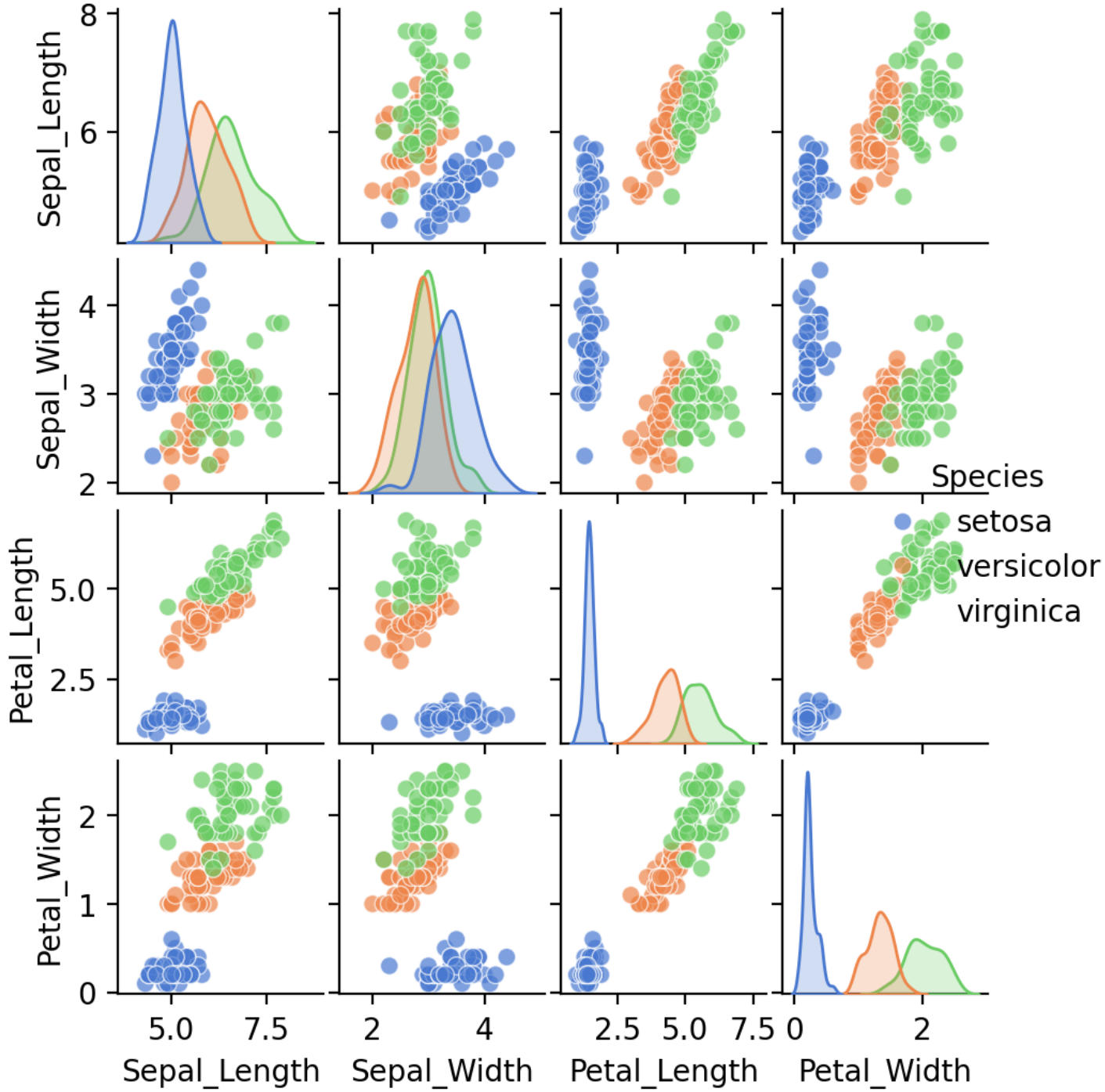
- The scatter plot effectively illustrates how Sepal Length and Sepal Width vary across different species of the Iris dataset, providing insights into the distribution and potential clustering of the species based on these two features.



Query5: @data_viz_agent visualize the loaded dataset as seaborn pairplot of the features data with the following parameters kind="scatter", diag_kind="kde", hue="Species", palette="muted", plot_kws={'alpha':0.7}. Adjust the size of the figure to be 5 by 5 inches.

The data_viz_agent Replies:

- The pairplot provides a comprehensive view of the relationships between all features in the dataset, allowing for easy identification of patterns and distributions among the different species of iris flowers.



Using the Production Plant Dataset

	Date	Baseline Period	Actual Consumption (kWh)	Avg WB Temperature (°F)	Product A	Product B	Product C	Seasonal Indicator	Max (AvgWB,55) (°F)	Total Production	Expected Consumption (kWh)	Actual Minus Expected (kWh)	Cumulative Energy Savings (kWh)
0	1/1/2014	1,000,000	53,757.39	53.14	135,249.76	273,359.78	248,526.1658	0	55	408,609.53	65,195.52	11,438.13	0
1	1/2/2014	1,000,000	47,778.01	51.45	0	0	16,653.4008	0	55	0	50,894.19	3,116.17	3,116.17
2	1/3/2014	1,000,000	70,153.83	41.39	147,185.53	346,725.65	293,444.1101	0	55	493,911.18	68,181.08	-1,972.75	1,143.42
3	1/4/2014	1,000,000	72,805.76	35.97	249,292.6	349,968.46	340,708.9425	0	55	599,261.06	71,868.32	-937.43	205.99
4	1/5/2014	1,000,000	70,841.87	36.07	245,493.57	343,220.03	435,672.3103	0	55	588,713.6	71,499.16	657.29	863.28

- This dataset reflects an energy consumption (kWh) of a production plant (Refrigerated Warehouse) facility for 3 types of products (Product A, Product B and Product C).
- The energy consumption strongly follows the seasonal and temperature changes.
- This strongly suggests the following multivariate (4 variables) linear regression dependence.

Expected Consumption (kWh) = A+ B*Avg Temperature + C*Total Production +D*Seasonal Indicator

Task:

- Consider column "Expected Consumption (kWh)" as dependent variable y of the following 3 independent variables: "Avg WB Temperature (°F)", "Total Production" and "Seasonal Indicator".

  y = "Expected Consumption (kWh)"

  x1 = "Avg WB Temperature (°F)"

  x2 = "Total Production"

  x3 = "Seasonal Indicator"

- Determine the coefficients A, B, C and D by perform Multi-variable Linear Regression using the following linear equation formed from the columns of the dataset.

$$y = A + B*x1 + C*x2 + D*x3$$

Query1:

This dataset relates the energy consumption (kWh) of a Refrigerated Warehouse facility for 3 types of products (Product A, Product B and Product C).
Consider column "Expected Consumption (kWh)" as dependent variable y of the following 3 independent variables: "Avg WB Temperature (°F)", "Total Production" and "Seasonal Indicator".

y = "Expected Consumption (kWh)"
x1 = "Avg WB Temperature (°F)"
x2 = "Total Production"
x3 = "Seasonal Indicator"

Determine the coefficients A, B, C and D by perform Multi-variable Linear Regression using the following linear equation formed from the columns of the dataset.

$$y = A + B*x1 + C*x2 + D*x3$$

Then visualize the columns "Expected Consumption (kWh)" and the dependent variable y determined by the linear regression coefficients versus the column "Date" as Plotly line plot.

Results returned by the App:

A = 3.708e+04

B = 318.9330

C = 0.0352

D = 3620.4546

OLS Regression Results

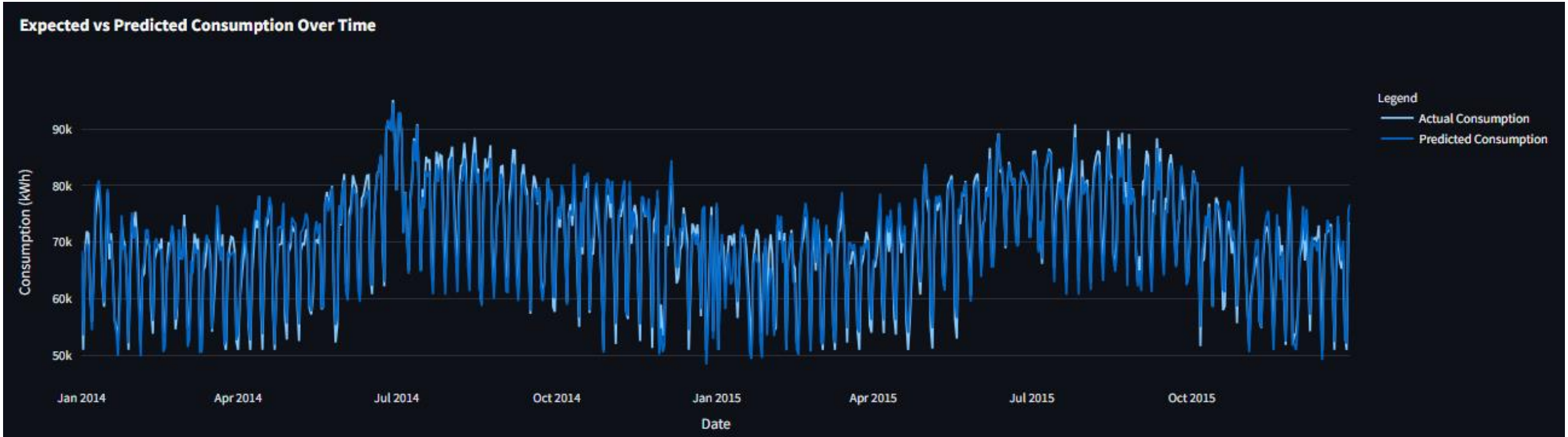
Dep. Variable:	Expected Consumption (kWh)	R-squared:	0.953
Model:	OLS	Adj. R-squared:	0.952
Method:	Least Squares	F-statistic:	4865.
Date:	Wed, 13 Nov 2024	Prob (F-statistic):	0.00
Time:	13:38:05	Log-Likelihood:	-6615.9
No. Observations:	730	AIC:	1.324e+04
Df Residuals:	726	BIC:	1.326e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
const	3.708e+04	370.600	100.048	0.000	3.64e+04	3.78e+04
Avg WB Temperature (°F)	318.9330	5.938	53.710	0.000	307.275	330.591
Total Production	0.0352	0.000	100.209	0.000	0.035	0.036
Seasonal Indicator	3620.4546	294.758	12.283	0.000	3041.775	4199.135

Omnibus:	7.633	Durbin-Watson:	0.580
Prob(Omnibus):	0.022	Jarque-Bera (JB):	6.665
Skew:	0.168	Prob(JB):	0.0357
Kurtosis:	2.674	Cond. No.	2.34e+06

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.34e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Query1: Visualization of the Expected and Predicted (regression model) energy consumptions.



Intercept (A): 37077.80154715449

Coefficient for Avg WB Temperature (B): 318.9330090843505

Coefficient for Total Production (C): 0.035247529411497835

Coefficient for Seasonal Indicator (D): 3620.4546296915983

A

- A